



OPEN Rethinking VLMs and LLMs for image classification

Avi Cooper¹, Keizo Kato^{2,4}, Chia-Hsien Shih^{1,3}, Hiroaki Yamane², Kasper Vinken¹, Kentaro Takemoto², Taro Sunagawa², Hao-Wei Yeh², Jin Yamanaka¹, Ian Mason^{1,4} & Xavier Boix¹

Visual Language Models (VLMs) are now increasingly being merged with Large Language Models (LLMs) to enable new capabilities, particularly in terms of improved interactivity and open-ended responsiveness. While these are remarkable capabilities, the contribution of LLMs to enhancing the longstanding key problem of classifying an image among a set of choices remains unclear. Through extensive experiments involving seven models, ten visual understanding datasets, and multiple prompt variations per dataset, we find that, for object and scene recognition, VLMs that do not leverage LLMs can achieve better performance than VLMs that do. Yet at the same time, leveraging LLMs can improve performance on tasks requiring reasoning and outside knowledge. In response to these challenges, we propose a pragmatic solution: a lightweight fix involving a relatively small LLM that efficiently routes visual tasks to the most suitable model for the task. The LLM router undergoes training using a dataset constructed from more than 2.5 million examples of pairs of visual task and model accuracy. Our results reveal that this lightweight fix surpasses or matches the accuracy of state-of-the-art alternatives, including GPT-4V and HuggingGPT, while improving cost-effectiveness.

Many of the best methods for visual representation learning now make use of Visual Language Models (VLMs)^{1–3}. These models combine vision and language (e.g. with contrastive training) in order to learn general vision-language representations. Recently, several new VLMs have been developed that combine VLMs with pre-trained Large Language Models (VLM+LLMs)^{4–6}. Since LLMs are trained on large corpora of text and computer code, combining pre-trained LLMs with VLMs has brought many benefits to VLMs, including the addition of outside knowledge and reasoning, iterative interaction, and an ability to provide open-ended responses.

While these new abilities are impressive, image classification tasks in which the image is categorized into a closed set of possible choices, such as in classic object recognition, constitute one of the primary goals of VLMs and computer vision in general. At the moment, it is unclear whether the additional capabilities of VLM+LLMs also contribute to an overall improvement to image classification. In the literature on VLM+LLMs, the evaluation focus has primarily centered on assessing the new abilities introduced by LLMs, i.e. open-ended answering and interactivity, with limited attention given to evaluating their impact on classifying the image into a given set of possible choices as it was traditionally done in image classification. Thus, in this work we re-evaluate VLM+LLMs on a diverse set of visual tasks with the goal of answering: when does leveraging LLMs improve model performance in traditional image classification setup?

We evaluate the performance of existing open source VLMs and VLM+LLMs across a range of benchmarks designed to test object and scene recognition as well as visual reasoning and outside knowledge. There are three possible outcomes to this experiment that *a priori* are not possible to discern:

1. VLM+LLMs are superior in all the tasks for image classification. This result may be the most expected, as VLM+LLMs currently dominate state-of-the-art accuracy in most vision tasks.
2. VLMs outperform VLM+LLMs depending on datasets. If we observe this, it is worth investigating when VLMs are better than VLM+LLMs.

¹We use the term LLM as it is commonly used, i.e. an auto-regressive text-generating model trained exclusively on text data. To distinguish VLMs that do and do not leverage an LLM as part of their architecture, we use the terms VLM+LLM and VLM respectively.

¹Fujitsu Research of America, Santa Clara, CA, USA. ²Fujitsu Limited, Kawasaki, Japan. ³University of Illinois at Urbana-Champaign, Urbana, IL, USA. ⁴Avi Cooper, Keizo Kato, Ian Mason and Xavier Boix have contributed equally to this work. ✉email: kato.keizo@jp.fujitsu.com

3. VLMs always surpass VLM+LLMs on image classification tasks. This result sounds contrary to expectations at first. However, we argue this is possible since unnecessary linguistic knowledge may interfere with the visual representations.

Results show that despite increasing parameter counts, VLM+LLMs are less performant than VLMs on object and scene recognition, while being superior on reasoning and outside knowledge tasks. In a series of experiments, we show that these results are consistent over seven models, ten datasets and multiple prompt variations. Our results are illustrated with examples in Fig. 1.

These observations show that different models excel at different vision tasks. We therefore introduce a computationally efficient system to route the task to the best model. To do so, we fine-tune GPT-2 to act as a router using the results from our experiments (about 2.5 million examples of pairs of visual task and model accuracy) and evaluate the router on held-out tasks. Because of this specialized training, our approach demonstrates a remarkable effectiveness in held-out visual tasks — it outperforms HuggingGPT⁷, and matches the accuracy of GPT-4V⁸ while improving in cost-effectiveness. Therefore, a lightweight LLM can effectively be trained to arbitrate between models and, based on user input, select one that is most suitable for the task at hand.

Do VLM+LLMs always beat VLMs?

In this section we analyze the capabilities of VLM+LLMs across multiple image classification tasks. We first introduce the VLMs and VLM+LLMs that we analyze and then introduce the evaluation methodology, which includes an overview of the datasets and prompting strategies used. Finally, we present the findings derived from our analysis.

Comparing VLM+LLMs with VLMs

To fairly compare the effect of leveraging LLMs for vision-language processing, we compare VLM+LLMs and VLMs that have exactly the same visual representations. That is, that use identical vision encoders. In this way we assess the effectiveness of different strategies for handling text, one that uses LLMs and the other that does not.

Note that including an LLM in a VLM necessitates the inclusion of additional machinery around the LLM to integrate it with the vision encoder (fusion modules, training objective, additional data, etc.). Two prevalent strategies exist in designing VLM+LLM models, namely that visual information can be incorporated into the LLM via embeddings (e.g. Flamingo⁵) or via text descriptions of the image (e.g. PNP-VQA⁴). We investigate models from each approach.

Flamingo vs. CLIP The architecture of Flamingo⁵ consists of the pre-trained and frozen vision encoder from CLIP and an LLM, connected with trainable attention layers. Flamingo is trained on a mixture of large-scale multimodal data from the web. We use the open-source OpenFlamingo⁹ implementation, utilizing CLIP ViT-L/14¹ as the vision encoder and MPT¹⁰ as the LLM. Thus, we directly compare Flamingo with CLIP as they have the same vision encoder but different techniques for handling text. To understand the impact of LLM size

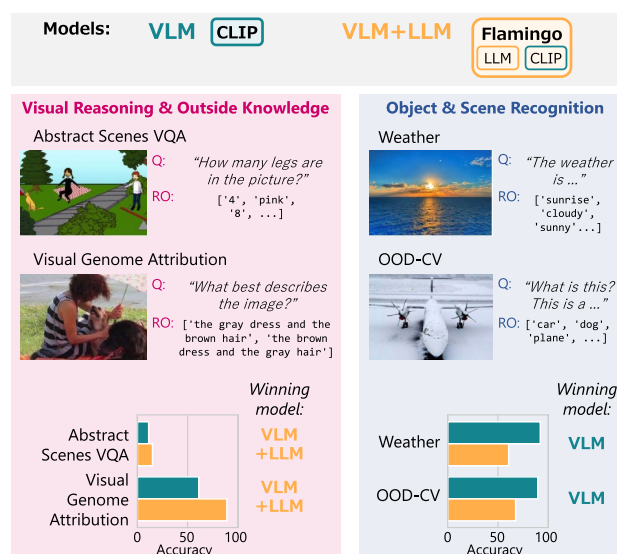


Fig. 1. VLMs achieve higher performance on object and scene recognition tasks than VLM+LLMs with the same vision encoder. Q: prompt provided to the models. RO: response options provided to the models. Left: on tasks involving visual reasoning and outside knowledge the VLM+LLM (Flamingo) outperforms the VLM (CLIP) with the same vision encoder. Right: on object and scene recognition the VLM is superior to the VLM+LLM. Note that Flamingo only uses CLIP's vision encoder, but since this vision encoder is pre-trained with the full CLIP architecture, in the figure we show CLIP as part of Flamingo. Pictures used in this figure come from the dataset explained in the "Appendix A".

on performance, we conducted experiments with Flamingo employing different parameter LLM configurations, including 3 billion (3b) and 9 billion (9b) parameters.

PNP-VQA vs. BLIP PNP-VQA⁴ facilitates communication between pre-trained and frozen models through language, without any additional training. First, PNP-VQA uses a VLM, namely BLIP², to identify the relevant parts in the image for the task at hand and then generates relevant captions (again, with BLIP) for each of the identified parts. Then an LLM, based on UnifiedQA¹¹, provides a final answer by taking into account the prompt along with the captions. Since PNP-VQA makes use of BLIP internally, directly comparing them provides insights into handling text with and without leveraging LLMs.

LiT vs. CLIP and Flamingo Locked-image text Tuning (LiT)³ is a VLM with the same vision encoder as CLIP and Flamingo but a larger text encoder than CLIP. It is a VLM and not a VLM+LLM as the text encoder is trained in a contrastive manner between text and image features, rather than leveraging pre-trained LLMs. We test two variations of LiT text encoder: BERT-base with 110M parameters (LiT-B/16) and BERT-large with 340M parameters¹² (LiT-L/16), which are much larger than the text encoder in CLIP (vanilla Transformer with 63M parameters¹). We include LiT in our evaluation to assess whether the differences in performance between VLMs and VLM+LLMs come from the pre-trained LLM rather than from simply having larger text encoders to handle text.

Returning to closed-ended evaluation

In language modelling tasks, such as translation and image captioning, evaluation is typically open-ended where models engage in open-ended text generation. On the other hand, image classification is inherently a closed-ended task, requiring a selection from a set of options rather than general description. In the open-ended case, evaluation is unclear because the degree of detail required is somewhat ambiguous. It is not sufficient to say that there is a tree in the image — the model must decide whether it is an elm, oak or willow tree if the task requires so. Thus, we adopt closed-ended evaluation for all experiments.

The methods we employ to compute closed-ended predictions differ across model types. For VLMs, the prediction of the model is the closed-ended option whose representation is best aligned with the image representation through a dot product. For VLM+LLMs, like PNP-VQA and Flamingo, the logits of each of the closed-ended options are used to compute a sequence probability. The sequence of logits with the highest average probability is the prediction of the model. As we show below in the results, differences in how models handle closed-ended prediction does not affect the overall conclusions.

Zero-shot visual tasks

LLMs excel at zero-shot tasks¹³ and can capture nuanced relationships and contextual information that might not be explicitly present in visual data. Conceivably, the extensive knowledge encapsulated by LLMs could be harnessed to address visual tasks when there is a lack of training data.

Zero-shot visual tasks allow us to assess the effectiveness of the synergy between VLMs and LLMs in a more controlled manner than if there would be task-specific training samples for fine-tuning. To avoid adding confounding factors arising from fine-tuning design choices (like deciding which layers to unfreeze) all experiments in this paper revolve around zero-shot visual tasks. We next introduce the 10 zero-shot datasets we leverage for our analysis, each characterized by varying degrees of reasoning complexity and knowledge requirements (see "Appendix A" for further details).

CIFAR-100¹⁴—a widely used benchmark for general object recognition. **OOD-CV**¹⁵—an out-of-distribution object classification, where objects undergo diverse pose, shape, occlusion and texture variations. **Weather**¹⁶—a multi-class weather recognition from images. **Skin Cancer**¹⁷—diagnosis for melanoma detection. **Hateful Memes**¹⁸—hatred identification in memes through a combination of image and text analysis. **ScienceQA**¹⁹—contains questions with images about scientific topics that require outside knowledge and reasoning capabilities. It resembles a high-school science exam. **Visual Genome Relation**²⁰—given an image and a relation of the form “X relation Y”, this benchmarks test whether the model can pick the correct order “X relation Y”, instead of “Y relation X”. **Visual Genome Attribution**²⁰—evaluates the ability to attribute properties to objects appropriately, structured in a similar fashion as the Visual Genome Relation dataset. **Abstract Scenes VQA**²¹—a commonly used dataset to assess the ability to answer questions related to high-level semantic and abstract information from scenes. **Binary Abstract Scenes**²²—the same as Abstract Scenes VQA but the response options are restricted to ‘yes’ and ‘no’ in order to remove biased responses.

Note that we do not use ImageNet or Visual Genome for object recognition datasets as some VLMs use them for training their vision encoders and it is unclear whether these datasets can be effectively used to assess zero-shot capabilities.

Prompt choice

It is well known that prompt choice can have a significant impact on model performance^{23–28}, and work has been conducted to specifically improve LLM’s reasoning capabilities, through Chain-of-Thought²⁹, Tree of Thought³⁰ and similar processes.

During evaluation we observed that varying the prompt dramatically affected the evaluation performance of the various networks. For example, when evaluating CLIP on CIFAR-100, using prompts of the form “What is this? This is aquarium_fish” or “What is this? This is beaver” (with class names inserted as-is from the CIFAR-100 dataset), resulted in an accuracy of 44.6%. We modified some of the class names, separating words and using simpler forms where appropriate (for example, “aquarium_fish” to “aquarium fish” and “aeroplane” to “plane”). We also prepended the appropriate article (a/an) in object classification tasks. The result was prompts of the form “What is this? This is an aquarium fish” and “What is this? This is a beaver.” These changes alone increased accuracy of CLIP on CIFAR-100 up from 44.7 to 69.1%.

	CIFAR-100	OOD-CV	Weather	Skin Cancer	Hateful Memes	ScienceQA	Visual Genome Attribution	Visual Genome Relation	Abstract Scenes VQA	Binary Abstract Scenes
Chance Rate	1.0	16.1	31.9	57.1	58.7	36.0	50.1	50.2	5.72	50.8
VLM BLIP	64.4 / 66.6	84.7 / 89.2	80.2 / 86.7	56.1 / 57.1	48.4 / 53.0	39.3 / 40.2	79.4 / 79.6	54.9 / 56.8	16.6 / 19.2	49.3 / 49.6
VLM+LLM PNP-VQA	51.8 / 54.7	75.3 / 78.1	58.7 / 73.5	50.1 / 57.1	46.6 / 54.3	49.8 / 50.9	77.9 / 77.9	58.2 / 58.2	42.0 / 50.5	57.8 / 58.1
VLM CLIP	60.1 / 70.4	89.4 / 90.9	92.0 / 95.1	54.7 / 57.1	51.4 / 57.9	47.2 / 49.3	61.0 / 62.6	54.1 / 54.4	11.1 / 13.6	51.4 / 51.9
VLM+LLM Flamingo-3B	48.3 / 68.1	66.0 / 88.4	54.4 / 82.7	57.2 / 57.1	50.0 / 58.7	50.6 / 52.8	87.0 / 88.3	60.3 / 60.6	22.2 / 31.5	49.2 / 49.2
VLM+LLM Flamingo-9B	53.2 / 64.3	72.8 / 87.7	64.2 / 87.2	58.2 / 62.2	50.0 / 58.7	49.2 / 51.3	89.0 / 90.8	60.6 / 61.2	14.6 / 18.9	49.2 / 49.2
VLM LiT-B/16	74.8 / 77.8	86.1 / 87.6	87.8 / 91.6	54.8 / 57.1	50.7 / 53.6	37.2 / 41.2	65.5 / 66.5	51.9 / 52.1	6.9 / 8.0	49.5 / 50.2
VLM LiT-L/16	74.3 / 78.4	83.9 / 85.3	88.3 / 92.0	55.3 / 62.2	52.5 / 56.3	37.4 / 40.1	65.1 / 65.2	53.2 / 53.2	8.2 / 9.7	49.6 / 49.9

Table 1. Accuracies of VLMs and VLM+LLMs across datasets (left: average across prompts / right: best prompt accuracy).

A VLM+LLM and its constituent VLM are color-coded, with the result for the better performing model underlined for each dataset. The highest accuracy achieved for each dataset is in bold. The first four datasets are object and scene recognition where VLM models are almost always superior to VLM+LLMs. The final six columns show tasks requiring reasoning and outside knowledge on which VLM+LLMs outperform VLMs.

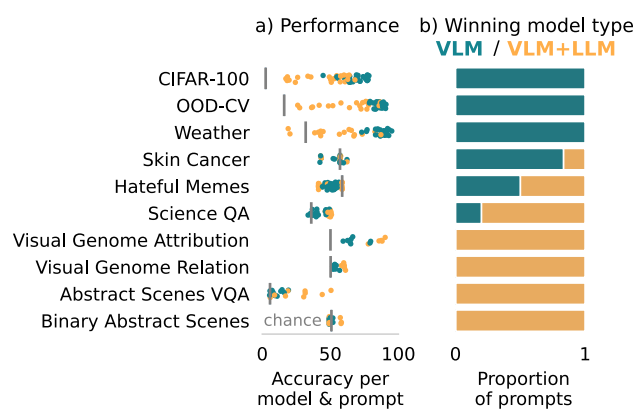


Fig. 2. Input prompts affect model performance but mostly do not affect winning model type. (a) Model performance for each model and prompt combination (markers) per dataset (y-axis). The accuracy varies across prompts, in particular for VLM+LLM models in recognition tasks (yellow markers in the first three rows). (b) Proportion of times a VLM or VLM+LLM wins, across all prompts. Most datasets (y-axis) show a proportion of 1 for either VLM or VLM+LLM, indicating the winning model type is not affected by the specific prompt.

We also found that alternative phrasing of the question could impact performance. For example, on the Abstract VQA dataset, prompting with the form “Is the dog asleep?” yielded an accuracy of 8.3% in one Flamingo model, while prompting with the form “Using the image, the answer to Is the dog asleep? is most likely” yielded an accuracy of 29.3% for the same model.

For this reason, we evaluated all models on a range of prompt formulations appropriate for each dataset. A full description of the class name alterations and prompt variations can be found in “Appendix B”.

Results

We analyze results on 462 experiments, including ten datasets, seven models and between two to nine prompt strategies per dataset. Results are displayed in Table 1 and Fig. 2. Table 1 reports the average accuracy among prompts and also the best accuracy among prompts, for each model and dataset. We highlight in boldface the best model per dataset, and underline the best model among each group of VLM+LLM and corresponding VLM. In order to ensure that the winning models displayed in Table 1 are largely prompt-independent, in Fig. 2 we depict the dependency of our results on the prompting strategy. Concretely, in Fig. 2a, we display the distribution of accuracy across prompt variations, indicating with different colors whether the model was a VLM or a VLM+LLM. As expected, there is substantial variability in accuracy among models across different prompt variations, sometimes more than 50%. Despite this high variability, Fig. 2b shows the proportion of times a VLM or VLM+LLM obtains higher accuracy for each prompt variation. We observe consistent model type across datasets, which highlight that the trends are prompt-independent.

The key trends that we observe by analyzing the results in Table 1 and Fig. 2 are twofold:

1. For most datasets that require visual reasoning and outside knowledge, VLM+LLMs achieves superior accuracy (see Fig. 2b and last six columns in Table 1). VLM+LLMs showcase superior capabilities in extracting high-level semantic information and adapting to conceptual understanding, which may be an expected re-

- sult. Note that the superior performance of VLM+LLMs cannot be attributed to the fact that the VLM+LLMs contain more parameters to process text. Despite the fact that LiT leverages a text encoder with many more parameters than CLIP's, CLIP achieves superior results to LiT on visual tasks that require reasoning. Thus, parameter count alone does not explain the superior performance of Flamingo over CLIP.
2. For most datasets of object and scene recognition, VLMs consistently outperform their corresponding VLM+LLMs (see Fig. 2b and the first four result column in Table 1), i.e. BLIP outperforms PNP-VQA and CLIP outperforms Flamingo. Note that this result cannot be attributed to the possibility that VLM+LLMs might struggle with closed-ended evaluation, as VLM+LLMs still excel in closed-ended visual tasks that require reasoning, as discussed before in point 1. We also observe that comparing the two versions of Flamingo, i.e. with the LLMs with 3B and 9B parameters, provides some improvements in some cases, but it is not sufficient to outperform VLMs. All this suggests that the reduction in accuracy may be attributed to the way the LLM is integrated with the vision encoder.

It is notable that all models struggle in some of these benchmarks, with accuracies close to chance, specifically for the Hateful Memes and Skin Cancer datasets. This highlights the need for further improvement in zero-shot visual tasks, despite the significant strides in recent years.

Regarding the best-performing model among all, there is no clear winner. For recognition datasets, CLIP and LiT dominate. In datasets that require a higher degree of reasoning and outside knowledge, Flamingo and PNP-VQA excel depending on the dataset. Thus, exploring the integration of these models into a unified system becomes a key research direction, allowing us to leverage the strengths of each model for optimal performance in all scenarios. In the next section, we pursue this idea by introducing a computationally efficient solution that combines the strengths of all the models into one system.

A fix: routing the task to the best model

Consider the following set up: a user provides a machine learning system with an image and a question about the image, and the response options. In Sect. 2 we have seen that VLMs and VLM+LLMs complement each other in the tasks they excel at. Since there is no single best model, we aim to design a system that can select the right model based on the user input. This capability opens up two exciting possibilities:

1. *Enhanced Generalization through Model Diversity*: The router enables a hybrid integration of models that specialize in different modalities or tasks, allowing the system to achieve superior performance by utilizing the most capable model for each specific input. The router's ability to combine outputs from diverse models increases the system's robustness and generalization capability. By selecting models that offer complementary perspectives on the input data, the router reduces the risk of overfitting or bias associated with a single model's limitations. This is particularly valuable in tasks with high variability or ambiguity, such as natural language understanding or complex visual scene interpretation.
2. *Resource-Efficient AI Deployments*: In resource-constrained environments, such as mobile devices or edge computing, the router can optimize computational efficiency by selecting lightweight models when high precision is not required, and more powerful models when accuracy is critical. This flexible approach balances performance and resource consumption, making it ideal for real-world deployment scenarios.

To implement this model selection system, we propose to pass the user input to an LLM which then learns to route the input to the best model for the user's task. Figure 3 shows an overview of this approach.

Training an LLM to act as a router

Previous LLM-based model selection systems make use of human written descriptions of the model capabilities⁷ or synthetically generated data³¹. In contrast, we use real model execution results calculated during our experiments (i.e. the experiments done to create Table 1). Importantly, this means our router learns from experience with actual data rather than some possibly inaccurate proxy as in previous works.

After running experiments over all models, datasets and prompts we create a dataset of more than 2.5 million samples containing (i) an input prompt and image, (ii) a set of options to respond with, (iii) model performance. All of the input is represented in natural language. For the input image, we calculate simple image metadata in the form of image resolution and per channel mean and standard deviations which are prepended to the prompt. The set of response options is a list of possible answers to the visual task. We find that the router performs best when the list of response options are not included as input to the LLM router, even though these are necessary to execute the predicted model for closed-ended evaluation. Per sample model performance is represented firstly by a boolean that is true if the sample is correctly classified by the model and false otherwise. Secondly, for each sample we additionally include average model performance over each dataset. "Appendix D" shows the full format of our data.

This initial dataset is then filtered by first removing any samples that use a prompt strategy that is not valid across all models (see "Appendix B"). By including in the training set different variations of prompts, we encourage the router to become robust to prompt variations. Following this, for a given input prompt and image we compare performance across all models to select the single best model, first by discarding any model that did not correctly classify the input and then selecting the model with the best average performance for the prompt on the dataset from the models remaining. In the case where all models incorrectly classify an image, we also select the model with best average performance. After this filtering process we are left with a training dataset containing approximately 280,000 training samples.

As we want our router to select a model based on the user's natural language query, we require an LLM. In order to keep the routing process relatively lightweight we pick a pre-trained GPT-2³² to use as a router. We then

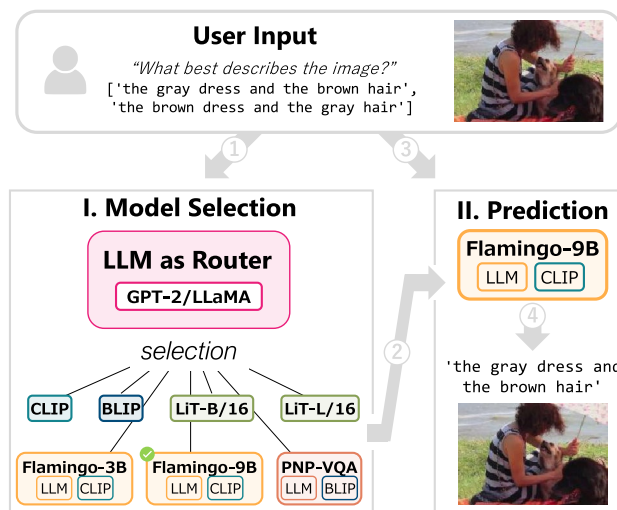


Fig. 3. Our router approach where an LLM selects a suitable VLM/VLM+LLM to obtain high accuracy. The user input is first provided to the LLM router (1), which uses this information to select a model. The chosen model (2) is then provided with the user input (3) and generates a prediction (4). Pictures used in this figure come from the dataset explained in the "Appendix A".

fine-tune GPT-2 taking the input prompt and image metadata as input, and the correct model as output, which is predicted as the next token after the input. The fine-tuning is done using the standard cross-entropy loss. Other implementation details are shown in "Appendix D". We also experiment with the router architecture by fine-tuning LLaMa³³ to assess whether our method is base model agnostic.

Evaluating on held-out datasets

To test the generalization abilities of the router on new tasks, we test on unseen datasets in a cross-validation style. In this way, we test the ability of the router to generalize to datasets where accuracy measurements are not available during training. Thus, we train ten separate models, each time holding out one of the ten datasets which is then used to evaluate the performance of the router. Thus, by testing the LLM router on out-of-distribution datasets, we can properly evaluate its generalization capabilities.

We initially compare the performance of our LLM router against basic model selection techniques both in terms of accuracy on unseen data and computational cost, expressed as the number of models that must be run in order to use the technique. These model selection baselines are the following:

Chance—no models are run and a random label is assigned to each image. *Average*—we report the average performance over all models, note that in the limit this is the same as randomly sampling a model each time a new input is received. *Voting*—all models are run and the modal output is selected, this can be seen as a form of model ensemble. *Oracle*—requires knowing the average accuracies for all models on the held-out dataset and then selecting the best performing model for a given input. *Upper bound*—per image oracle that shows how often at least one of the available models correctly classifies a given input.

In Table 2 we see that our router outperforms all baselines, achieving the best performance on the largest number of datasets (seven of ten) as well as the best average performance. Most notably our method outperforms the strongest baseline of ensemble voting, whilst being more computationally efficient at inference time. Ensemble voting requires executing every model on the inputs, making it linear in the number of models. On the other hand, our method only executes the single model chosen by the router, so is of constant order. Compared to the best baseline with the same computational cost (*average*), our method achieves almost a ten-percentage point improvement on average. Regarding the baselines that require access to ground truth results on the held-out dataset (*oracle* and *upper bound*), our method is very close to the oracle performance, suggesting our router often picks the best model despite not having access to ground truth data. Finally, the upper bound column shows that by collating only a relatively small number of models we can theoretically achieve extremely high performance if the perfect model could be picked for every individual image, which indicates that there is still room for future improvements.

Comparison with state-of-the-art

We additionally compare against two modern state-of-the-art solutions that are applicable to our use case. Firstly HuggingGPT⁷ which uses prompt design and in-context learning to get GPT-3¹³ to provide a model, or set of models, from the HuggingFace Library³⁴ to execute the task and then summarize the results. Secondly, we consider the newly released GPT-4V(ision)^{8,35}. We use GPT-4V to evaluate a very recent approach which, motivated by scaling laws^{36,37}, focuses on massive compute and data as a solution for improving visual understanding. Whilst technical details about GPT-4V are relatively limited we do know that this system is substantially more compute heavy than our relatively small LLM router and open-source models, any one of which can run on a single V100 GPU. Details of our implementations of these methods can be found in "Appendix C".

	Chance	Average	Voting	Router (GPT-2)	Oracle	Upper Bound
CIFAR-100	1.0	60.9	76.5	60.7	72.7	92.3
OOD-CV	16.1	81.4	89.5	85.9	89.3	97.5
Weather	31.9	70.9	84.1	84.1	86.4	96.4
Skin Cancer	57.1	41.7	40.8	48.7	50.0	86.3
Hateful Memes	58.7	49.4	48.4	49.7	53.4	98.2
ScienceQA	36.0	44.7	44.0	57.7	56.6	92.5
VG Attribution	50.1	75.3	80.6	88.8	91.6	99.8
VG Relation	50.2	55.8	57.9	60.4	60.4	97.3
Abstract Scenes	5.7	17.2	16.9	42.3	42.3	70.2
B. Abstract Scenes	50.8	51.4	52.3	58.6	58.5	92.1
Average	35.7	54.9	59.1	63.7	66.1	92.9
Average (Weighted)	18.0	55.5	61.2	64.9	68.4	89.6
Cost (N. of Models)	-	$\mathcal{O}(1)$	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$

Table 2. Performance comparison for different model selection baselines. Accuracy is averaged across prompt strategies. Each row evaluates a separately trained router which sees no samples from the listed evaluation dataset during training. As well as surpassing other methods in terms of accuracy our router also has constant, rather than linear, computational cost in the number of models. We report average performance over datasets as well as the average weighted by the different sizes of the datasets.

	HuggingGPT	GPT-4V	Router (GPT-2)	Router (LLaMA)
CIFAR-100	15.0	55.0	60.0	60.0
OOD-CV	75.0	82.5	77.5	72.5
Weather	62.5	91.7	95.8	75.0
ScienceQA	57.5	65.0	50.0	47.5
Skin Cancer	38.1	38.1	61.9	61.9
Hateful Memes	42.5	45.0	65.0	65.0
Visual genome attribution	87.5	90.0	87.5	85.0
Visual genome relation	65.0	62.5	57.5	57.5
Abstract scenes VQA	37.5	50.0	30.0	27.5
Binary abstract scenes	50.0	77.5	62.5	57.5
Average	53.1	65.7	64.8	60.9
Average (Weighted)	53.4	66.0	63.6	60.3

Table 3. Comparison against state-of-the-art models on a separate smaller test set. Our method outperforms HuggingGPT and is competitive with GPT-4V with much lower computational cost.

Table 3 compares our LLM routing approach against these baselines. We note that due to rate restrictions and high monetary costs ("Appendix C") this table evaluates a total of 365 samples randomly selected (40 per dataset, except Weather and Skin Cancer, which have 24 and 21, respectively). Whilst this limited number of samples means we must be careful not to read too much into small differences in performance for individual datasets, some interesting trends emerge overall.

When comparing to HuggingGPT, our approach outperforms HuggingGPT on nine of ten datasets and by more than ten percent absolute accuracy on average. Although HuggingGPT has access to many more models than our system, its reliance on text descriptions of the models and in-context learning means it fails to perform as well as our approach which is trained on actual model accuracy. What's more, due to the careful prompt engineering and in-context samples, queries to HuggingGPT run for several thousand tokens which at current pricing for GPT-3 (text-davinci-003) means we faced an inference cost of approximately \$0.05 per sample.

When comparing to GPT-4V our method performs competitively with the average performance difference being less than one percent, this is despite our method requiring vastly less computational resources². For instance, the largest VLM + LLM model in our experiment, Flamingo-9B, consists of 9 billion parameters. Even when augmented with a router powered by GPT-2 or LLaMA (15 billion/13 billion parameters, respectively), the total parameter count remains significantly smaller than that of GPT-4V. While the router could add latency as it adds an additional step to generate the response, this latency is negligible because we use a relatively small model as a router (GPT-2).

²Very conservatively assuming GPT-4V is at least as big as GPT-3 this means it has a minimum of 175 billion parameters. In contrast our largest models use around 10 billion parameters.

We also evaluated an LLM router with LLaMA as the LLM. However, we did not observe an improvement of accuracy, possibly because more training data is needed given LLaMA has more parameters than GPT-2. In "Appendix E" we show further results with the LLaMA router.

The success of our LLM router approach can be attributed to its specific training on a dataset that includes pairs of visual tasks and model accuracy. In contrast, other approaches rely on proxies like human-generated descriptions of the models.

Analyses and ablations

In this section we further analyze our results, first by exploring how the router makes decisions on held-out datasets and then by conducting an ablation study where we experiment with the input information provided to the router.

To explore how the router makes decisions we plot how often each model is selected by the router for each held-out dataset in Fig. 4a. We can then compare this to how often each model is used for each dataset in the training data in Fig. 4b. From this figure, we obtain the following insights about the decision-making process of the router. If we consider the dataset OOD-CV, we see that LiT-B/16 is always selected by the router. Note that this is the best model for CIFAR-100. Since OOD-CV is not seen during the training of the router, results suggest that the router perceives the unseen OOD-CV data is most similar to the input from CIFAR-100 and as a result, uses the best model for CIFAR-100 when presented with OOD-CV samples. We see similar patterns for other datasets like Visual Genome Attribution and Relation. This figure shows that the router can transfer knowledge between similar datasets due to the similarity in prompts for similar tasks.

We additionally provide an ablation study using the GPT-2 router in Table 4. We evaluated different input formats for the router (i) with or without the response options (RO) (ii) with or without image metadata (MD). Firstly, eliminating response options from the input prompt largely improves the performance. Thus, although we expected that including response options would help to find similar datasets in the training data, it seems that response options hinder rather than help. One possible explanation is that the response options are very specific to each dataset and act as shortcuts that lead to overfitting rather than generalization across datasets.

Recall that as input to the LLM router, we include simple metadata from the image (see Sect. 3.1). Table 4 shows (a small) improvement due to the use of the image metadata. The dataset which has the largest performance gap when metadata is ablated is Weather (84.1% vs. 73.9%). This difference in performance suggests there may be image-level (rather than dataset-level) patterns that can be used to improve routing; an interesting avenue for further investigation is multimodal routers with stronger image processing abilities. See "Appendix E" for a more in-depth analysis.

Finally, in Table 4, we show in-distribution performance, where the router is trained on all datasets and evaluated on held-out samples from datasets it has seen during training. To do so, we created an 80/10/10 train/validate/test split. Here, we see that even when datasets are held-out the router performs similarly to the in-distribution case, suggesting our router has learned to generalize well across datasets. A similar ablation study for LLaMA along with further heat maps are shown in "Appendix E".

Related work

Foundation models: VLMs & LLMs

Foundation models³⁸ have changed the landscape of vision and language, with large image^{39–41}, language^{13,42,43} and explicitly multimodal^{5,6,35,44–46} transformer based models⁴⁷ creating new capabilities in terms of generalization and low-shot performance⁴⁸. Of these various models, our work is most concerned with Vision Language Models^{1–3} and methods that combine VLMs with LLMs^{4,5,49}.

Some recent intriguing findings highlight the nuanced relationship between LLMs and VLMs^{5,50,51}. In the study by Roth et al.⁵⁰, CLIP evaluated on images with randomly generated query texts demonstrates comparable

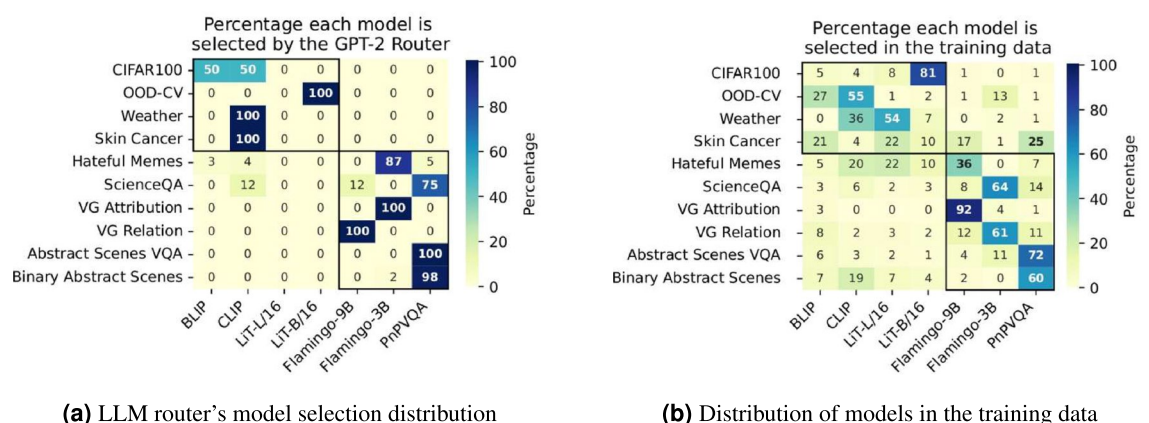


Fig. 4. Heat maps of the LLM router's model selection distribution and the distribution of models in the training data.

	w/ MD		w/o MD		InD w/ MD
	w/ RO	w/o RO	w/ RO	w/o RO	w/o RO
CIFAR-100	56.8	60.7	56.6	60.7	72.7
OOD-CV	85.9	85.9	85.9	85.9	89.4
Weather	85.2	84.1	86.4	73.9	75.0
Skin cancer	48.7	48.7	48.7	47.4	48.7
Hateful Memes	49.0	49.7	49.9	50.4	53.7
ScienceQA	52.4	57.7	52.9	57.3	56.0
Visual Genome Attr.	88.8	88.8	89.0	88.8	88.8
Visual Genome Rel.	60.4	60.4	60.4	60.4	58.5
Abstract scenes VQA	16.1	42.3	25.8	38.5	42.7
Binary abstract scenes	58.5	58.6	58.5	58.8	58.5
Average	60.2	63.7	61.4	62.2	64.4
Average (Weighted)	57.3	64.9	59.9	63.9	68.6

Table 4. Ablation Study. We experiment with different input formats for the LLM router testing with and without image metadata (MD) and with and without the set of possible response options (RO). The first four columns evaluate on held-out datasets, for comparison, we also show the in-distribution case (InD) where all datasets are seen during training. Accuracy is averaged across prompt strategies.

performance with CLIP evaluated on images with LLM-generated query texts. This is further evidence, complementary to ours, that how best to use LLMs to improve VLMs is remains an open question.

Meanwhile, a careful examination of the Flamingo paper, reveals that it lags behind state-of-the-art VLM models in ImageNet and Kinetics700 classification tasks as shown in "Appendix B" in⁵. This trend can also be seen by comparing Tables 5 and 19 in the results of PaLI⁶. Although several large multimodal models have been introduced since Flamingo, their primary advancements lie in aspects such as increased size and expanded modality coverage. The core architecture and fusion methodologies, however, remain largely consistent with earlier designs. For instance, while IDEFICS⁵² is a larger-scale model compared to Flamingo, its architecture is fundamentally identical to that of Flamingo. Similarly, Gemini⁵³, a state-of-the-art multimodal model, handles a variety of modalities-including image, text, audio, and video. While transformer layers are employed to fuse features across these modalities, there is no explicit mechanism to address the issue we have discussed. Our findings makes this observation explicit and general. By focusing on the difference in performance of VLM+LLMs and their corresponding VLMs on classification tasks we reveal VLM+LLMs' limitations across multiple datasets and models and then, we propose a solution to it.

Routing, model selection & augmented LLMs

Model selection is a standard problem in machine learning⁵⁴ but is most commonly examined in the case where validation data is used to select the best single model by estimating test error for future i.i.d. test data. In our case we instead use other datasets to learn a model which can dynamically select the best model for a given task, which we refer to as routing. Most similar to our work is concurrent work on routing for language benchmarks⁵⁵, which has interesting links to meta-learning⁵⁶. Yet, the results of Shnitzer et al.⁵⁵ are limited to only natural language tasks.

Many tool augmented LLMs^{57–61}, that is LLMs that can reference external tools, can be viewed as performing routing to enable tasks beyond their initial training data. Several systems have been proposed to allow LLMs to reference external machine learning models however, these works are either aspirational in nature⁶², use synthetic data itself generated by an LLM³¹, or rely on prompt engineering and in-context learning^{7,63}. In contrast, our method for routing learns cost-effective strategies from accuracies obtained by running and evaluating models, which serve as training data for the LLM router.

Some visual programming studies^{63,64} have attempted to use LLMs to transform complex visual problems into ones that are easier for VLMs to solve. These works demonstrate how LLMs can analyze and decompose questions, enabling the completion of complex tasks by integrating components like object detectors, VLMs, VLM+LLMs, and other specialized systems, rather than relying exclusively on a single model. These studies do not challenge the contribution we present in our paper-namely, that VLMs and VLM+LLMs possess distinct strengths depending on the type of question. Consequently, incorporating VLMs and VLM+LLMs as an option, such as through a router, in these previous works remains a promising approach for enhancing performance.

Conclusions

We found that VLM+LLMs were better at advanced reasoning and knowledge tasks but were worse on object and scene recognition tasks when compared with VLMs with the same vision encoder. To address this issue, we proposed the use of a low-resource LLM that serves as a router, intelligently selecting the most suitable model among VLMs and VLM+LLMs for each visual task query. This LLM router strategy achieved higher accuracy than other model selection and ensemble baselines, and is superior or on par with several other state-of-the-art approaches, like HuggingGPT and GPT-4Vision, which use far more computational resources. Our LLM router's effectiveness is rooted in its specialized training on a corpus of examples, consisting of pairs of visual tasks and

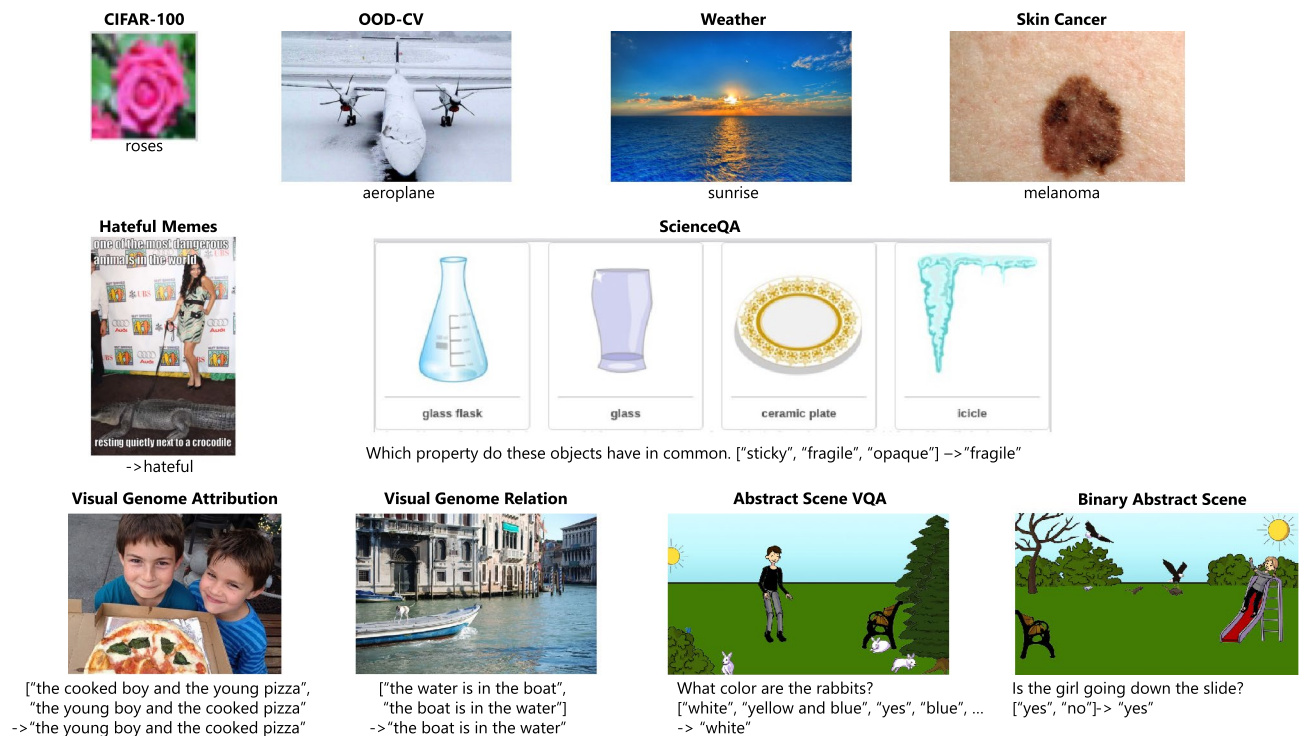


Fig. 5. Example samples from the datasets used to evaluate VLMs and VLM+LLMs. Images in this figure come from the datasets described in "Appendix A", captions are the ground truth labels..

model accuracy. In contrast, competing approaches use proxies such as human-generated descriptions of the models.

The LLM router solution employs an LLM to address issues arising from the leveraging of LLMs to improve VLMs, i.e. an LLM is used to solve an issue that stems from another LLM. The effectiveness of the router solution indicates that LLMs alone have the potential to always augment the VLM capabilities across all visual tasks. Therefore, future investigations aimed at leveraging LLMs for VLMs should focus on identifying more effective strategies for integrating them, as suggested by the success of our LLM router solution.

Limitations

This study has a number of limitations that future work could build upon. First, since this paper focuses on routing we limited the study to zero-shot model evaluation; it needs to be seen whether the conclusions of this paper extend beyond zero-shot regime. Also, while the models tested were state-of-the-art models at the time when we ran the experiments for this study, the field of AI advances very rapidly and new models have already surpassed the capabilities of the models tested in this study. It is unclear if the conclusions of this study would generalize to newer models. Finally, we have only tested the router in held-out datasets, and thus, it is unclear how users in practical applications would assess it.

Data availability

The datasets used and/or analysed during the current study available from the corresponding author on reasonable request. The detail is described in "Appendix A".

Appendix A: Datasets

We utilize multiple datasets with distinct characteristics to evaluate the performance of our proposed model. The datasets can be divided into two categories, tasks of *Object & Scene Recognition* (i.e., CIFAR-100, OOD-CV, Weather, and Skin Cancer) and those of *Visual Reasoning & Outside Knowledge* (i.e., Hateful Memes, ScienceQA, Visual Genome Attribution, Visual Genome Relation, Abstract Scenes VQA, and Binary Abstract Scenes.). When evaluating zero-shot model performance we use the test sets of the datasets to reduce the chance that the models we evaluate have seen the images during training. We show example images from datasets in Fig. 5. Original images for each dataset can be accessed through the provided URL.

CIFAR-100 The CIFAR-100 dataset¹⁴ consists of 60,000 32x32 pixel color images. Each image contains an object from one of 100 classes. This dataset is widely utilized for general object recognition tasks. We adhere to the same train and test split as originally proposed. The test split contains 10,000 images.<https://www.cs.toronto.edu/~kriz/cifar.html>

OOD-CV OOD-CV¹⁵ is a benchmark dataset introduced to enhance the evaluation of vision algorithms' robustness in real-world scenarios. The dataset includes out-of-distribution examples across 10 object categories,

with variations in pose, shape, texture, context, and weather conditions. It enables benchmarking models on tasks including image classification, object detection, and 3D pose estimation. The dataset is composed of images from PASCAL3D+ and additional images collected and annotated by the creators. We combine *phase-1* and *phase-2* for a total of 21,502 images in the test set. <https://github.com/wufeim/OOD-CV-Data>

Weather The Multi-class Weather Dataset¹⁶ is designed to test classification capabilities on images of various weather conditions. It consists of a total of 1,125 images, categorized into four classes: Sunrise, Shine, Rain, and Cloudy. The test split is 226 randomly sampled images (20% of the dataset). <https://data.mendeley.com/dataset/s/4drtyfjfy/1>

Skin cancer While prior studies have applied models like CLIP in the medical domain, specifically for diagnosing conditions such as COVID-19 and pneumonia from chest X-rays⁶⁵, these images differ significantly from visible light RGB images. The skin cancer dataset¹⁷, which focuses on melanoma detection natural images, is an excellent task candidate. The dataset is maintained by the University of Waterloo. It consists of images extracted from public databases DermIS and DermQuest, with response options healthy/cancerous and manual cropping to center the lesion in the image. Since this is the smallest dataset, we sample 95% of the dataset for evaluation, resulting in 196 images for testing. <https://vip.uwaterloo.ca/skin-cancer-detection/>

Hateful memes The task in the Hateful Memes dataset¹⁸ is to determine whether a meme is hateful or non-hateful. The dataset is designed to enforce multimodality — correctly classifying samples based on image or text alone is (essentially) impossible since individually the image or text may seem innocuous but combined they convey a mean or hateful message. 10,000 memes (image with text written on top) are collected/generated. We combine *test_seen* and *test_unseen* as test dataset, resulting in 3,000 images in total. <https://ai.meta.com/tools/hatefulmemes/>

ScienceQA The ScienceQA dataset¹⁹ is a benchmark consisting of approximately 21,000 multimodal, multiple-choice questions spanning a diverse set of science topics, including abstract conceptual diagrams. Each question is annotated with corresponding lectures and explanations, providing a rich context for assessment. The dataset is designed to test the multi-hop reasoning ability of AI systems. We exclude any samples from the test split that do not contain images, resulting in 2,017 images. <https://github.com/lupantech/ScienceQA>

Visual genome attribution & relation The Visual Genome Attribution & Relation dataset²⁰ is crafted from the Visual Genome dataset⁶⁶ and is a crucial part of the Attribution and Relation (AR) benchmark. This benchmark is designed to assess the capabilities of vision and language models in understanding and processing complex relationships and attributes within images. The dataset is structured to challenge models with tasks such as determining the correct order in relations (e.g., “X relation Y” vs “Y relation X”) and accurately attributing properties to objects. It includes 23,937 cases related to relations and 28,748 cases focused on attributes. When tested on this dataset, existing models have shown deficiencies in the complex reasoning required to succeed at this task. The test split contains 5,736 images for Visual Genome Attribution and 4,753 images for Visual Genome Relation. <https://homes.cs.washington.edu/~ranjay/visualgenome/api.html>

Abstract scenes VQA and binary abstract scenes This study utilizes Abstract Scenes, a part of the VQA v1 dataset²¹, which we refer to it as Abstract Scenes VQA. The images are characterized by simplified, clip-art style representations, which tests the generalization capabilities of VQA models. The tasks focuses on high-level semantics in vision and language, ensuring that both are essential for accurate responses. We use 30,000 test images for Abstract Scenes VQA and 11,327 test images for Binary Abstract Scenes. https://visualqa.org/vqa_v1_download.html

Appendix B: Prompt variations

As discussed in Sect. 2.4, varying the prompt had a significant impact on model performance.

Note that we employ closed-ended evaluation in this study, as discussed in Sect. 2.2. Therefore, when we say we prompt the model, what we mean is that we provide it with a set of text prompts that consists of a single question combined with each of the options (from the set of response options).

Prompt grammar

In order to methodically explore a range of different prompts, we outlined a small grammar to compose prompts. The grammar is as follows:

- *key*: Datasets have several text fields, including the class name and additional text-context. One value that is used across datasets is *class_name*. When this value is used, *n* prompts are generated, where *n* is the number of classes in the dataset (for ex, 100 in the case of CIFAR-100) and each prompt is set to one of the class names. Some datasets have dataset-specific keys
- *a_an*: An argument that adds an a/an appropriately to the key
- *rename_classes*: If the key text has a defined substitution, the key will be swapped for its rename
- *arg*: Can be one of [*a_an*, *rename_classes*]
- *{[arg]* | + key}*: The tag is fully described by zero or multiple *args*, a character (which can be omitted if no *args* are specified) and the key. Additional text can surround the tags.

Further, each dataset defines a set of questions and options. If a set of options is not defined, this means that the options are provided by the dataset for each sample. A prompt is formed by concatenating a question to an option formulation. Sometimes we leave the question empty to test whether the class name alone is the best prompt. The cartesian product between all question and option formulations defines the full set of prompts. Below we outline the prompts, options and additional text fields for each dataset.

Prompts for each dataset

CIFAR-100

- Question Formulations
 - “”
 - “This is ”
 - “What is this? This is ”
- Option Formulations
 - “{class_name}”
 - “{rename_classes class_name}”
 - “{a_an, rename_classes class_name}”
- Class Renames
 - aquarium_fish: aquarium fish
 - pickup_truck: pickup truck
 - lawn_mower: lawn mower
 - sweet_pepper: pepper
 - maple_tree: maple
 - oak_tree: oak
 - palm_tree: palm
 - pine_tree: pine
 - willow_tree: willow
- OOD-CV Has the same question and option formulations as CIFAR-100Class Renames
 - aeroplane: plane
 - diningtable: table

Weather

- Question Formulations
 - “”
 - “It is ”
 - “The weather is ”
- Option Formulations
 - “{class_name}”
 - “{rename_classes class_name}”
- Class Renames
 - Sunrise: sunrise

	w/ MD		w/o MD	
	w/ RO	w/o RO	w/ RO	w/o RO
CIFAR100	72.7	72.7	71.5	71.5
OODCV	89.1	89.4	92.0	92.0
Weather	77.3	75.0	84.6	84.6
Skin cancer	48.7	48.7	36.8	42.1
Hateful memes	55.0	53.7	60.0	60.0
ScienceQA	58.6	56.0	56.0	59.5
VisualGenomeAttribution	91.6	88.8	94.5	94.5
VisualGenomeRelation	58.5	58.5	61.0	60.5
AbstractScenesVQA	42.3	42.7	46.5	45.0
BinaryAbstractScenes	58.5	58.5	54.0	54.5
Average	65.2	64.4	65.7	66.4
Average (Weighted)	68.4	68.6	66.8	67.2

Table 5. GPT-2: Ablation Study for the *in-distribution* case where all datasets are seen during training. We experiment with different input formats for the LLM router testing with and without image metadata (MD) and with and without the set of possible response options (RO). Accuracy is averaged across prompt strategies.

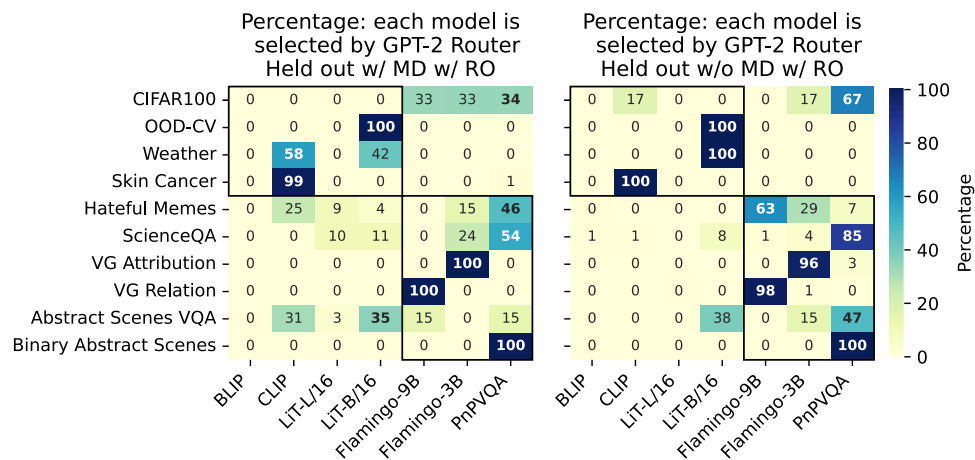


Fig. 6. GPT-2 Router, model selection across datasets and prompt strategies. Left: with metadata and with response options vs Right: without metadata and with response options.

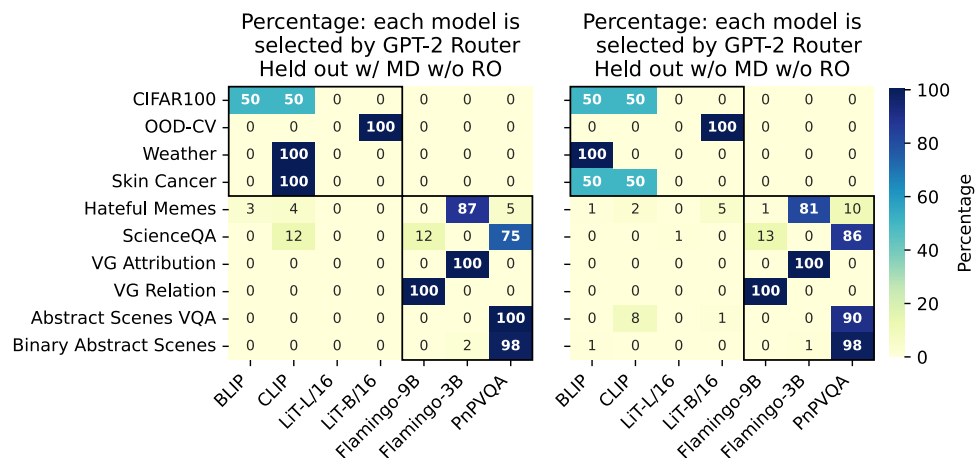


Fig. 7. GPT-2 Router, model selection across datasets and prompt strategies. Left: with metadata and without response options vs Right: without metadata and without response options.

- Cloudy: cloudy
- Shine: sunny
- Rain: rainy

Skin Cancer

- Question Formulations
 - “”
 - “This is ”
 - “This skin is ”
- Option Formulations
 - “{class_name}”
 - “{rename_classes class_name}”
- Class Renames
 - melanoma: cancerous
 - notmelanoma: healthy

Hateful Memes

	w/ MD		w/o MD	
	w/ RO	w/o RO	w/ RO	w/o RO
CIFAR-100	52.5	64.5	58.8	66.1
OOD-CV	89.4	84.7	82.4	88.0
Weather	62.5	76.1	73.9	86.4
Skin cancer	50.0	43.4	52.6	42.1
Hateful memes	50.8	51.5	48.8	49.5
ScienceQA	58.5	52.2	54.8	54.8
Visual genome attribution	91.0	90.0	91.4	89.7
Visual genome relation	62.1	63.1	61.5	61.5
Abstract Scenes VQA	34.2	44.2	16.9	38.9
Binary abstract scenes	54.8	60.5	62.5	61.1
Average	60.6	63.0	60.4	63.8
Average (Weighted)	62.4	65.8	57.1	65.9

Table 6. Ablation Study for LLaMA (Held out). We experiment with different input formats for the LLM router testing with and without image metadata (MD) and with and without the set of possible response options (RO). Like GPT-2 it's evaluated on held-out datasets. Accuracy is averaged across prompt strategies.

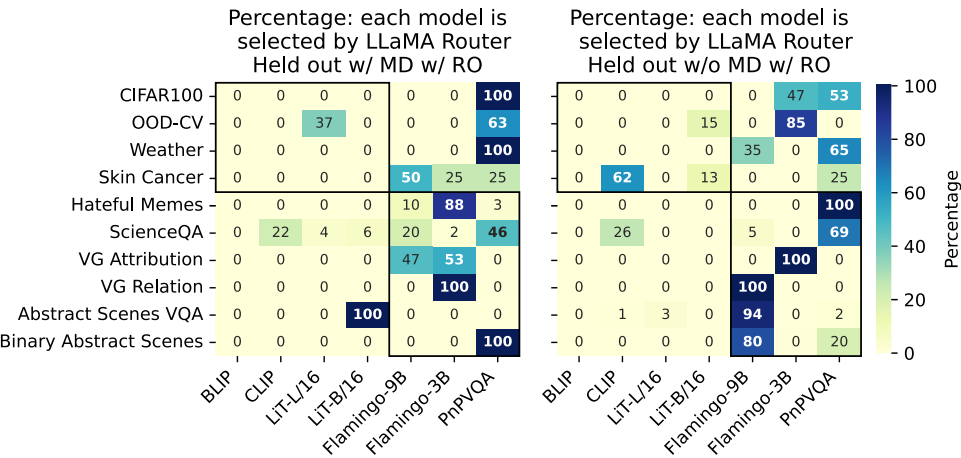


Fig. 8. LLaMA Router, model selection across datasets and prompt strategies. Left: with metadata and with response options vs right: without metadata and with response options.

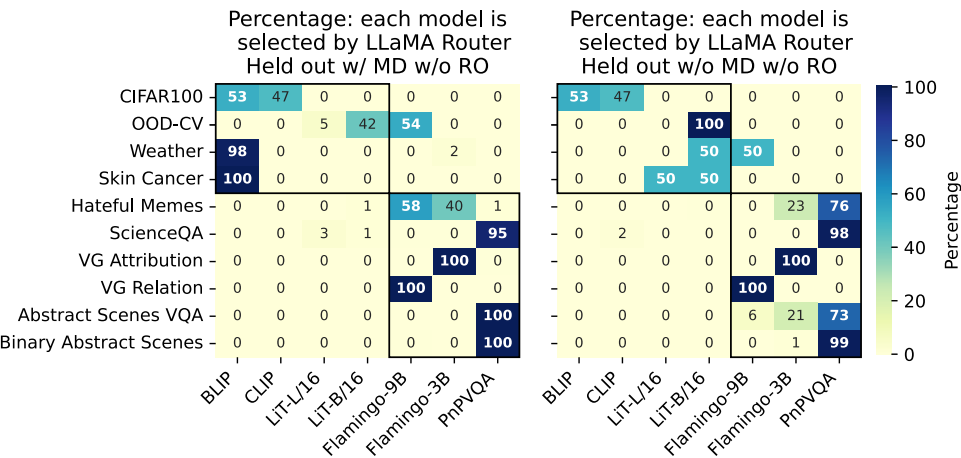


Fig. 9. LLaMA Router, model selection across datasets and prompt strategies. Left: with metadata and without response options vs right: without metadata and without response options.

Below, the key *text* is provided by the dataset for each sample. It is the text-string of the text written on the meme.

- Question Formulations
 - “”
 - “{*text*}. ”
 - “{*text*}. This meme is ”
 - “This is an image of a meme. It contains the text: {*text*}. The meme is ”
- Option Formulations
 - “{*class_name*}”
 - “{*rename_classes* *class_name*}”
- Class Renames
 - not mean: nice
 - mean: mean

ScienceQA

Below, the keys *class_question* and *class_hint* are provided by the dataset for each sample. *class_question* is the question for each sample, while *class_hint* is some additional (unnecessary but helpful) context for each sample provided by the dataset.

- Question Formulations
 - “”
 - “{*class_question*} ”
 - “{*class_hint*} {*class_question*} ”
 - “Question: {*class_hint*} {*class_question*} ”
 - “{*class_hint*} Question: {*class_question*} ”

Visual genome attribution & relation

Below, the key *class_question* is provided by the dataset for each sample. It is the question for each sample.

- Question Formulations
 - “”
 - “{*class_question*} ”
 - “This best describes the image: ”

Abstract scenes VQA and binary abstract scenes

Below, the key *class_question* is provided by the dataset for each sample. It is the question for each sample.

- Question Formulations
 - “”
 - “Question: {*class_question*} Answer: ”
 - “Using the image, the answer to {*class_question*} is most likely ”

Prompting PNP-VQA

The implementation of PNP-VQA requires a question to serve as the prompt, separately from the options. However for several datasets, we tried prompts that have an empty question, denoted with “”. In these cases, we provide the empty string to PNP-VQA, to ensure that evaluation is the same across models. However PNP-VQA cannot provide a response in these cases. For the experiments in Table 1 we use all the prompts described in this section. When training the planner, to ensure all models have the same number of prompts and to avoid prompt/model dependence, we remove this question from further analysis for all models.

Appendix C: Baseline implementation

This section describes how the baselines GPT-4V and HuggingGPT were implemented to create a fair comparison with our LLM router.

GPT-4V(ision)

GPT-4V(ision) was released for developer access on 11/6/23. Our experiments took place between 11/13/23 and 11/16/23. During this period GPT-4V was in preview, meaning organizations could only make 100 API requests per day, which limited the number of images we were able to test with it.

Further, though the API documentation provides for options that might be used to approximate close-ended evaluation, we were unable to make these parts of the API perform as documented for GPT4-V. We therefore took an alternative approach to simulate closed-ended evaluation. We provided the following system prompt: “Complete the prompt from the list called OPTIONS, enclosed by [and]. Your response can only include a

single element from this list”

and in the user message, we provided the prompt and the closed set of response options within brackets. For each dataset we pick the prompt that is valid for all models and achieved the highest accuracy in the experiments to create Table 1.

To evaluate GPT-4V each completion was determined to be one of:

- *Correct Prediction*: if the response of the model contained only one element from OPTIONS and it was the correct label.
- *Incorrect Prediction*: if the response of the model contained only one element from OPTIONS and it was an incorrect label.
- *No Guess*: if the response contained no elements from OPTIONS
- *Multiple Guesses*: if the response contained multiple elements from OPTIONS To calculate the accuracies in Table 3 we considered *Incorrect Predictions*, *No Guess* and *Multiple Guesses* as errors. This means the accuracy was calculated as the number of correct predictions divided by the total number of samples.

To more accurately measure the performance of GPT-4V we allow ten completions from the model. We select the most common prediction (correct or incorrect) from GPT-4V as the final prediction, only selecting *No Guess* or *Multiple Guesses* if all of the ten completions fail to make a valid prediction. This approximates choosing the most probable option from the closed set.

HuggingGPT

To run HuggingGPT we use the code made publicly available at <https://github.com/microsoft/JARVIS>, the code was pulled on 10/19/23, HuggingGPT uses text-davinci-003 as the LLM. Due to making multiple requests with long context prompts, we found HuggingGPT cost approximately \$0.05 per-sample, or around \$20 to calculate results on the small subset of data in Table 3 (excluding experimentation and development requests). We evaluated HuggingGPT in the same manner as GPT-4V, but calculated only one completion per sample due to these costs.

For the most part we used the default settings provided in the repository, making the following changes to the prompting to encourage the system to select from the set of valid response options.

- *System Prompt: Original*: “#4 Response Generation Stage: With the task execution logs, the AI assistant needs to describe the process and inference results.”
- *Ours*: “#4 Response Generation Stage: With the task execution logs, the AI assistant needs to provide the best completion for the prompt provided by the user.”
- *User Message: Original*: “Yes. Please first think carefully and directly answer my request based on the inference results. Some of the inferences may not always turn out to be correct and require you to make careful consideration in making decisions. Then please detail your workflow including the used models and inference results for my request in your friendly tone. Please filter out information that is not relevant to my request. Tell me the complete path or urls of files in inference results. If there is nothing in the results, please tell me you can’t make it.”
- *Ours*: “Yes. Please complete the prompt from the list called OPTIONS, enclosed by [and]. Your response can only include a single element from this list.” The user message is then additionally given the instruction:

“Using the image and the options provided please complete the following prompt. {prompt}... OPTIONS: {response options}”, where {prompt} and {response options} are replaced with the relevant prompt and response options for the given sample. As with GPT-4V, for each dataset we pick the prompt that is valid for all models and achieved the highest accuracy in the experiments to create Table 1.

We also found HuggingGPT was rarely able to actually access and run models from the Hugging Face Hub. Due to this limitation and to improve reproducibility we ran HuggingGPT in local mode, downloading all relevant implemented models. The models available locally to HuggingGPT were downloaded on 11/11/23. The full list of models we made available to HuggingGPT is:

- nlpconnect/vit-gpt2-image-captioning
- llyasviel/ControlNet
- llyasviel/sd-controlnet-canny
- llyasviel/sd-controlnet-depth
- llyasviel/sd-controlnet-hed
- llyasviel/sd-controlnet-mlsd
- llyasviel/sd-controlnet-openpose
- llyasviel/sd-controlnet-scribble
- llyasviel/sd-controlnet-seg
- runwayml/stable-diffusion-v1-5
- damo-vilab/text-to-video-ms-1.7b
- microsoft/speecht5_asr
- JorisCos/DCCRNNet_Libri1Mix_enhsingle_16k
- espnet/kan-bayashi_ljspeech_vits
- facebook/detr-resnet-101
- microsoft/speecht5_hifigan
- microsoft/speecht5_vc

- openai/whisper-base
- Intel/dpt-large
- facebook/detr-resnet-50-panoptic
- facebook/detr-resnet-50
- google/owlvit-base-patch32
- impira/layoutlm-document-qa
- ydshieh/vit-gpt2-coco-en
- dandelin/vilt-b32-finetuned-vqa
- lambdalabs/sd-image-variations-diffusers
- facebook/maskformer-swin-base-coco
- Intel/dpt-hybrid-midas
- Salesforce/blip-image-captioning-large
- facebook/maskformer-swin-large-ade
- microsoft/beit-base-patch16-224-pt22k-ft22k
- openai/clip-vit-large-patch14
- deepset/roberta-base-squad2
- google/tapas-base-finetuned-wtq
- distilbert-base-uncased-finetuned-sst-2-english
- gpt2
- Dataset: Matthijs/cmu-arctic-xvectors

Appendix D: Implementation detail

In this section, we provide additional implementation details for evaluations and experiments.

Closed-ended evaluation

In Sect. 2.2, we motivated our use of closed-ended evaluation. As mentioned, the implementation of closed-ended evaluation differed across models. Here we outline the implementation in various model architectures. This evaluation is computed for each sample in the dataset, where each sample is an image and the corresponding set of text prompts ($o \in O$): question and response options.

Contrastive models

This group includes CLIP, BLIP, and LiT. The model has two different encoders, one for images and one for text. For each sample, we extract the N -dimensional image embedding, $v_i \in \mathbb{R}^N$, and the text embedding matrix, $V_O \in \mathbb{R}^{|O| \times N}$, where rows of V_O are the N -dimensional embedding of the prompts $o \in O$. We then take the prediction to be the option indexed by $\text{argmax}(v_i^T \cdot V_O)$.

Language models

This group includes PNP-VQA, and Flamingo. Models of this type can return a log-probability for the next token, across all tokens in the vocabulary, given the preceding tokens. These probabilities are generally used for generating new text, but they can also be used for closed-ended evaluation. For each prompt, $o \in O$, we sum the log probabilities for each of the tokens in the prompt. The prediction is then the response option used to create the prompt which has the highest probability.

Data format

Section 3.1 describes the creation of training data for the LLM router. After this process a single sample in the dataset looks like

```
[img]dim:: (270, 317, 3) ave:: (23.1, 31.8,
46.2) std:: (15.1, 11.5, 10.9) [prompt]What
is this? This is ;;;['a car', 'a sofa',
'a train', 'a table', 'a chair',
'a boat', 'a plane', 'a motorbike',
'a bus', 'a bicycle' [SEP]model::
clip[response]correct::True;;;
avg_accuracy::0.88238
```

The values following the [img] marker are the resolution of the image along with per-channel mean and standard deviation of the pixel values. Then the prompt that should be completed is provided along with the set of response options (a car, a sofa etc.). These components are the possible inputs to the router that we experiment with. After the [SEP] token is a label for the model (CLIP). The final part is the boolean that marks whether or not the model gave the correct response for the image in questions as well as the average accuracy for the model on the dataset the image is from. The boolean and the average accuracy are used to select the best model for each image which is then used to train the LLM router as described in Sect. 3.1.

GPT-2 implementation

For GPT-2, we used the pre-trained model from <https://huggingface.co/gpt2>. The optimizer was Adam with an initial learning rate of 2×10^{-4} . The batch size was 1, we trained for 5000 iterations, after every 1000 iterations, an evaluation was done and the checkpoint with the best result was used as the final model.

LLaMA implementation

For LLaMA, we used a pre-trained model from <https://huggingface.co/baffo32/decapoda-research-llama-7B-hf>. LoRA with rank 8 and model precision of fp16 were applied to save computational cost. We optimized models with the AdamW optimizer with an initial learning rate of 1×10^{-3} . The batch size was 2. We trained for 500 iterations, an evaluation was done every 100 iterations and the checkpoint with the best result was used as the final model.

Appendix E: Further results

Further ablation studies for GPT-2

First, we examine the results of training the planner on all datasets and evaluating on unseen samples from those datasets, see Table 5. This is slightly different from holding out individual datasets as was done in the main text, instead this is ‘in-distribution’ evaluation. When compared with Table 4 we see that the router is only slightly better in this case despite being evaluated in-distribution. This shows that the LLM router described in the main text is able to generalize well to held-out datasets.

We also include heatmaps for the GPT-2 Router performance on held-out datasets with and without metadata and with and without response options, see Figs. 6, 7. Conclusions are the same as in the ablation study in Table 4. Note that routers trained with response options are more likely to choose an inferior model type (VLM or VLM+LLM) for a given dataset type (classification or reasoning). Instead, relying only on the prompts seems to make it easier for the router to determine the task type.

Next, we compare the routers trained with and without metadata, and without the response options. The router always selects CLIP as the best model when trained with metadata (Fig. 7, left), but when trained without metadata the router chooses CLIP and BLIP with equal probability (Fig. 7, right). Since CLIP is significantly better than BLIP on the Weather dataset, the router trained with metadata achieves higher performance. In the training data (Fig. 4, right), BLIP is selected by the router relatively often for the OOD-CV and Skin Cancer datasets. Since the images from Weather and Skin Cancer datasets look very different, it is likely they can be distinguished even with the simple metadata provided to the router, and therefore the router may learn to avoid using BLIP for such visually different images.

LLaMa ablation

We also include an ablation and heatmaps for the LLaMA router performance on held-out datasets with and without metadata and with and without response options, see Table 6 and Figs. 8, 9. Similarly to the GPT-2 router, the LLaMA router trained without response options performed better. On the other hand, when comparing performance with and without metadata, training without metadata achieves marginally higher average accuracy. As with GPT-2, by examining the heatmaps (Figs. 8, 9), we see that including response options tends to lead to less optimal model selection and hence lower performance.

Received: 28 October 2024; Accepted: 27 May 2025

Published online: 04 June 2025

References

- Radford, A. *et al.* Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 8748–8763 PMLR, (2021).
- Li, J., Li, D., Xiong, C. & Hoi, S. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning*, 12888–12900 PMLR, (2022).
- Zhai, X. *et al.* LiT: Zero-shot transfer with locked-image text tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 18123–18133 (2022).
- Tiong, A. M. H., Li, J., Li, B., Savarese, S. & Hoi, S. C. Plug-and-play VQA: Zero-shot VQA by conjoining large pretrained models with zero training. In Goldberg, Y., Kozareva, Z. & Zhang, Y. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2022*, 951–967, <https://doi.org/10.18653/v1/2022.findings-emnlp.67> Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, (2022).
- Alayrac, J.-B. *et al.* Flamingo: a visual language model for few-shot learning. *Adv. Neural. Inf. Process. Syst.* **35**, 23716–23736 (2022).
- Chen, X. *et al.* Pali: A jointly-scaled multilingual language-image model. In *The Eleventh International Conference on Learning Representations* (2023).
- Shen, Y. *et al.* Hugginggpt: Solving AI tasks with chatgpt and its friends in hugging face. arXiv preprint [arXiv:2303.17580](https://arxiv.org/abs/2303.17580) (2023).
- OpenAI. GPT-4 technical report. arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774) (2023).
- Awadalla, A. *et al.* Openflamingo: An open-source framework for training large autoregressive vision-language models. arXiv preprint [arXiv:2308.01390](https://arxiv.org/abs/2308.01390) (2023).
- Team, MN. Introducing mpt-7b: A new standard for open-source, commercially usable llms (2023). Accessed: 2023-05-05.
- Khashabi, D. *et al.* Unifiedqa: Crossing format boundaries with a single qa system. arXiv preprint [arXiv:2005.00700](https://arxiv.org/abs/2005.00700) (2020).
- Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805) (2018).
- Brown, T. *et al.* Language models are few-shot learners. *Adv. Neural. Inf. Process. Syst.* **33**, 1877–1901 (2020).
- Krizhevsky, A., Hinton, G. *et al.* Learning multiple layers of features from tiny images. <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf> (2009).
- Zhao, B. *et al.* OOD-CV: a benchmark for robustness to out-of-distribution shifts of individual nuisances in natural images. In *European Conference on Computer Vision*, 163–180 Springer, (2022).
- Ajayi, G. Multi-class weather dataset for image classification. Mendeley Data (2018). V1.
- Vision & Lab, I. P. Skin cancer detection. University of Waterloo (Available).
- Kiela, D. *et al.* The hateful memes challenge: Detecting hate speech in multimodal memes. *Adv. Neural. Inf. Process. Syst.* **33**, 2611–2624 (2020).

19. Lu, P. *et al.* Learn to explain: Multimodal reasoning via thought chains for science question answering. In Koyejo, S. *et al.* (eds.) *Advances in Neural Information Processing Systems*, vol. 35, 2507–2521 Curran Associates, Inc., (2022).
20. Yuksekgonul, M., Bianchi, F., Kalluri, P., Jurafsky, D. & Zou, J. When and why vision-language models behave like bags-of-words, and what to do about it? In *The Eleventh International Conference on Learning Representations* (2023).
21. Antol, S. *et al.* VQA: Visual Question Answering. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (2015).
22. Zhang, P., Goyal, Y., Summers-Stay, D., Batra, D. & Parikh, D. Yin and Yang: Balancing and answering binary visual questions. In *Conference on Computer Vision and Pattern Recognition (CVPR)* (2016).
23. Reynolds, L. & McDonell, K. Prompt programming for large language models: Beyond the few-shot paradigm. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–7 (2021).
24. Zhou, Y. *et al.* Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations* (2023).
25. Yang, Z. *et al.* An empirical study of gpt-3 for few-shot knowledge-based vqa. In *Proceedings of the AAAI Conference on Artificial Intelligence* **36**, 3081–3089 (2022).
26. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y. & Iwasawa, Y. Large language models are zero-shot reasoners. *Adv. Neural. Inf. Process. Syst.* **35**, 22199–22213 (2022).
27. Wang, X. *et al.* Self-consistency improves chain of thought reasoning in language models. arXiv preprint [arXiv:2203.11171](https://arxiv.org/abs/2203.11171) (2022).
28. Pitlis, S., Zhang, M. R., Wang, A. & Ba, J. Boosted prompt ensembles for large language models. arXiv preprint [arXiv:2304.05970](https://arxiv.org/abs/2304.05970) (2023).
29. Wei, J. *et al.* Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural. Inf. Process. Syst.* **35**, 24824–24837 (2022).
30. Yao, S. *et al.* Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint [arXiv:2305.10601](https://arxiv.org/abs/2305.10601) (2023).
31. Patil, S. G., Zhang, T., Wang, X. & Gonzalez, J. E. Gorilla: Large language model connected with massive apis. arXiv preprint [arXiv:2305.15334](https://arxiv.org/abs/2305.15334) (2023).
32. Radford, A. *et al.* Language models are unsupervised multitask learners. *OpenAI blog* (2019).
33. Touvron, H. *et al.* Llama: Open and efficient foundation language models (2023). [arXiv:2302.13971](https://arxiv.org/abs/2302.13971).
34. Wolf, T. *et al.* Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45 Association for Computational Linguistics, Online, (2020).
35. OpenAI. GPT-4V(ision) system card. *OpenAI blog* (2023).
36. Kaplan, J. *et al.* Scaling laws for neural language models. arXiv preprint [arXiv:2001.08361](https://arxiv.org/abs/2001.08361) (2020).
37. Zhai, X., Kolesnikov, A., Houlsby, N. & Beyer, L. Scaling vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 12104–12113 (2022).
38. Bommasani, R. *et al.* On the opportunities and risks of foundation models. arXiv preprint [arXiv:2108.07258](https://arxiv.org/abs/2108.07258) (2021).
39. Kirillov, A. *et al.* Segment anything. [arXiv:2304.02643](https://arxiv.org/abs/2304.02643) (2023).
40. Zou, X. *et al.* Segment everything everywhere all at once. arXiv preprint [arXiv:2304.06718](https://arxiv.org/abs/2304.06718) (2023).
41. Rombach, R., Blattmann, A., Lorenz, D., Esser, P. & Ommer, B. High-resolution image synthesis with latent diffusion models (2021). [arXiv:2112.10752](https://arxiv.org/abs/2112.10752).
42. Chowdhery, A. *et al.* Palm: Scaling language modeling with pathways. *J. Mach. Learn. Res.* **24**, 1–113 (2023).
43. Touvron, H. *et al.* Llama 2: Open foundation and fine-tuned chat models. arXiv preprint [arXiv:2307.09288](https://arxiv.org/abs/2307.09288) (2023).
44. Zhu, D., Chen, J., Shen, X., Li, X. & Elhoseiny, M. Minigpt-4: Enhancing vision-language understanding with advanced large language models. arXiv preprint [arXiv:2304.10592](https://arxiv.org/abs/2304.10592) (2023).
45. Li, J., Li, D., Savarese, S. & Hoi, S. BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML* (2023).
46. Driess, D. *et al.* Palm-e: An embodied multimodal language model. In arXiv preprint [arXiv:2303.03378](https://arxiv.org/abs/2303.03378) (2023).
47. Vaswani, A. *et al.* Attention is all you need. *Advances in neural information processing systems* **30** (2017).
48. Bubeck, S. *et al.* Sparks of artificial general intelligence: Early experiments with gpt-4. arXiv preprint [arXiv:2303.12712](https://arxiv.org/abs/2303.12712) (2023).
49. Huang, S. *et al.* Language is not all you need: Aligning perception with language models. arXiv preprint [arXiv:2302.14045](https://arxiv.org/abs/2302.14045) (2023).
50. Roth, K. *et al.* Waffling around for performance: Visual classification with random words and broad concepts. arXiv preprint [arXiv:2306.07282](https://arxiv.org/abs/2306.07282) (2023).
51. Zhang, Y., McKinzie, B., Shankar, V., Gan, Z. & Toshev, A. Pre-trained language models do not help auto-regressive text-to-image generation. In *NeurIPS Workshop. I Can't Believe It's Not Better (ICBINB): Failure Modes in the Age of Foundation Models* (2023).
52. Laurencon, H. *et al.* Obelics: An open web-scale filtered dataset of interleaved image-text documents. arXiv preprint [arXiv:2306.16527](https://arxiv.org/abs/2306.16527) (2023).
53. Gemini Team, R. A. e. a. Gemini: A family of highly capable multimodal models. arXiv preprint [arXiv:2312.11805](https://arxiv.org/abs/2312.11805) (2024).
54. Raschka, S. Model evaluation, model selection, and algorithm selection in machine learning. arXiv preprint [arXiv:1811.12808](https://arxiv.org/abs/1811.12808) (2018).
55. Shnitzer, T. *et al.* Large language model routing with benchmark datasets (2023). [arXiv:2309.15789](https://arxiv.org/abs/2309.15789).
56. Vilalta, R. & Drissi, Y. A perspective view and survey of meta-learning. *Artif. Intell. Rev.* **18**, 77–95 (2002).
57. Parisi, A., Zhao, Y. & Fiedel, N. Talm: Tool augmented language models. arXiv preprint [arXiv:2205.12255](https://arxiv.org/abs/2205.12255) (2022).
58. Schick, T. *et al.* Toolformer: Language models can teach themselves to use tools. arXiv preprint [arXiv:2302.04761](https://arxiv.org/abs/2302.04761) (2023).
59. Mialon, G. *et al.* Augmented language models: a survey. *Transactions on Machine Learning Research* (2023). Survey Certification.
60. Thoppilan, R. *et al.* Lamda: Language models for dialog applications. arXiv preprint [arXiv:2201.08239](https://arxiv.org/abs/2201.08239) (2022).
61. Suris, D., Menon, S. & Vondrick, C. Vipergpt: Visual inference via python execution for reasoning. arXiv preprint [arXiv:2303.08128](https://arxiv.org/abs/2303.08128) (2023).
62. Liang, Y. *et al.* Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. arXiv preprint [arXiv:2303.16434](https://arxiv.org/abs/2303.16434) (2023).
63. Gupta, T. & Kembhavi, A. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 14953–14962 (2023).
64. Lu, P. *et al.* Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems* **36** (2024).
65. Wang, Z., Wu, Z., Agarwal, D. & Sun, J. Medclip: Contrastive learning from unpaired medical images and text. arXiv preprint [arXiv:2210.10163](https://arxiv.org/abs/2210.10163) (2022).
66. Krishna, R. *et al.* Visual genome: Connecting language and vision using crowdsourced dense image annotations. *Int. J. Comput. Vision* **123**, 32–73 (2017).

Author contributions

A. Cooper, K. Kato, C. Shih, I. Mason and X. Boix conceptualized the research with contributions from K. Takemoto, H. Yeh and H. Yamane; A. Cooper, C. Shih and I. Mason wrote the code and ran the experiments in Sect. 2; K. Kato and I. Mason wrote the code and ran the experiments in Section 3; K. Takemoto, T. Sunagawa, H. Yeh and J. Yamanaka and H. Yamane contributed to the code to run the experiments; H. Yamane and K. Vinken

analyzed and visualized the data, with contributions from I. Mason and A. Cooper; H. Yamane imported the datasets with contributions from K. Takemoto; A. Cooper, K. Kato, C. Shih, I. Mason, K. Vinken and X. Boix wrote the paper with contributions from K. Takemoto and H. Yamane, H. Yeh; X. Boix ideated and supervised the research with contributions of I. Mason.

Declarations

Competing interest

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to K.K.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025